Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Шумихина М.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 08.11.26

Москва, 2026

Постановка задачи

Вариант 1.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Задача 1го Варианта: Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму.

Общий метод и алгоритм решения

Использованные системные вызовы:

- shm_open - создает и открывает объект разделяемой памяти (Shared Memory) в оперативной памяти;
- ftruncate - задает размер созданного объекта разделяемой памяти;
- mmap - отображает объект разделяемой памяти в виртуальное адресное пространство процесса;
- sem_open - создает и открывает именованный семафор;
- sem_wait / sem_post - захватывают и освобождают семафор для синхронизации доступа к памяти;
- fork - создает дочерний процесс;
- execv - запускает исполняемый файл клиента в дочернем процессе;
- waitpid - блокирует родительский процесс до завершения дочернего;
- getline - считывает строку с числами произвольной длины из стандартного ввода;
- strtol - преобразует строку в число с проверкой на переполнение;
- fopen / fprintf / fclose - используются дочерним процессом для записи результата в файл;
- munmap, close, sem_close, shm_unlink, sem_unlink - освобождают ресурсы и удаляют именованные объекты из системы.

Программа разделена на два исполняемых файла: родительский и дочерний. Основной процесс принимает имена разделяемой памяти и семафора через аргументы командной строки, создает объект разделяемой памяти через shm_open, задает ему размер и отображает в свое адресное пространство с помощью mmap. Затем создается именованный семафор с начальным значением 0, который блокирует доступ. Сервер считывает введенную пользователем строку с числами, записывает ее длину в первые байты разделяемой памяти и

копирует саму строку.

После этого сервер вызывает fork для создания дочернего процесса и с помощью execv запускает клиент, передавая ему имена объектов. Родитель вызывает sem_post, разрешая чтение, и ожидает завершения потомка через waitpid. Дочерний процесс подключается к уже существующим объектам, отображает память через mmap и ожидает сигнала sem_wait. Получив доступ, клиент считывает длину и строку из разделяемой памяти, разбивает ее на отдельные числа с помощью strtol, вычисляет их общую сумму и записывает результат в файл result.txt. В конце оба процесса освобождают ресурсы, а родитель удаляет объекты памяти и семафора из системы вызовами shm_unlink и sem_unlink.

Код программы

server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <errno.h>
#include <signal.h>


int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <shm_name> <sem_name>\n", argv[0]);
        return EXIT_FAILURE;
    }


    const char *shm_name = argv[1];
    const char *sem_name = argv[2];
```

```c
if (shm_name[0] != '/' || shm_name[1] == '\0' ||
    strchr(shm_name + 1, '/') != NULL ||
    sem_name[0] != '/' || sem_name[1] == '\0' ||
    strchr(sem_name + 1, '/') != NULL) {
    fprintf(stderr, "Names must start with '/' and contain no other '/'\n");
    return EXIT_FAILURE;
}

char *input_buf = NULL;
size_t input_capacity = 0;

ssize_t input_length = getline(&input_buf, &input_capacity, stdin);

if (input_length == -1) {
    perror("getline error");
    free(input_buf);
    return EXIT_FAILURE;
}

int shm_fd = -1;
char *shm_ptr = MAP_FAILED;
sem_t *sem = SEM_FAILED;

int shm_created = 0;
int sem_created = 0;

pid_t pid = -1;
```

```c
int exit_status = EXIT_FAILURE;


shm_fd = shm_open(shm_name, O_CREAT | O_EXCL | O_RDWR, 0600);


if (shm_fd == -1) {
    perror("shm_open error");
    goto cleanup;
}


shm_created = 1;


uint64_t data_length = (uint64_t)input_length;
size_t shm_size = sizeof(data_length) + (size_t)data_length;


if (ftruncate(shm_fd, (off_t)shm_size) == -1) {
    perror("ftruncate error");
    goto cleanup;
}


shm_ptr = mmap(NULL, shm_size,
               PROT_READ | PROT_WRITE,
               MAP_SHARED, shm_fd, 0);


if (shm_ptr == MAP_FAILED) {
    perror("mmap error");
    goto cleanup;
}
```

```
sem = sem_open(sem_name, O_CREAT | O_EXCL, 0600, 0);


if (sem == SEM_FAILED) {
    perror("sem_open error");
    goto cleanup;
}


sem_created = 1;


memcpy(shm_ptr, &data_length, sizeof(data_length));


memcpy(shm_ptr + sizeof(data_length),
       input_buf, (size_t)data_length);


pid = fork();


if (pid == -1) {
    perror("fork error");
    goto cleanup;
}


if (pid == 0) {
    sem_close(sem);
    munmap(shm_ptr, shm_size);
    close(shm_fd);
```

```
        char *args[] = {
            "./client",
            (char *)shm_name,
            (char *)sem_name,
            NULL
        };


        execv("./client", args);


        perror("execv error");
        _exit(EXIT_FAILURE);
    }


    if (sem_post(sem) == -1) {
        perror("sem_post error");
        kill(pid, SIGTERM);
        goto cleanup;
    }


    int status;
    pid_t result;


    do {
        result = waitpid(pid, &status, 0);
    } while (result == -1 && errno == EINTR);


    if (result == -1) {
        perror("waitpid error");
```

```
        goto cleanup;
    }


    if (WIFEXITED(status) && WEXITSTATUS(status) == EXIT_SUCCESS) {
        exit_status = EXIT_SUCCESS;
    } else {
        fprintf(stderr, "Сервер: клиент завершился с ошибкой.\n");
    }


cleanup:


    if (shm_ptr != MAP_FAILED) {
        munmap(shm_ptr,
               shm_created ? (size_t)sizeof(uint64_t) +
               (size_t)input_length : 0);
    }
    if (shm_fd != -1) {
        close(shm_fd);
    }
    if (sem != SEM_FAILED) {
        sem_close(sem);
    }
    if (sem_created) {
        sem_unlink(sem_name);
    }
    if (shm_created) {
        shm_unlink(shm_name);
    }
    free(input_buf);
    return exit_status;
```

```
}
client.c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <inttypes.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <semaphore.h>
#include <errno.h>
#include <limits.h>
#include <ctype.h>


int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <shm_name> <sem_name>\n", argv[0]);
        return EXIT_FAILURE;
    }


    const char *shm_name = argv[1];
    const char *sem_name = argv[2];


    int shm_fd = -1;
    char *shm_ptr = MAP_FAILED;
    sem_t *sem = SEM_FAILED;
    char *input = NULL;
```

```c
int exit_status = EXIT_FAILURE;
size_t shm_size = 0;

shm_fd = shm_open(shm_name, O_RDWR, 0);

if (shm_fd == -1) {
    perror("shm_open error");
    goto cleanup;
}

struct stat shm_info;

if (fstat(shm_fd, &shm_info) == -1) {
    perror("fstat error");
    goto cleanup;
}

if (shm_info.st_size < (off_t)sizeof(uint64_t)) {
    fprintf(stderr, "Invalid shared memory size\n");
    goto cleanup;
}

shm_size = (size_t)shm_info.st_size;

shm_ptr = mmap(NULL, shm_size,
               PROT_READ | PROT_WRITE,
               MAP_SHARED, shm_fd, 0);
```

```
if (shm_ptr == MAP_FAILED) {
    perror("mmap error");
    goto cleanup;
}

sem = sem_open(sem_name, 0);

if (sem == SEM_FAILED) {
    perror("sem_open error");
    goto cleanup;
}

while (sem_wait(sem) == -1) {
    if (errno != EINTR) {
        perror("sem_wait error");
        goto cleanup;
    }
}

uint64_t data_length;

memcpy(&data_length, shm_ptr, sizeof(data_length));

if (data_length > shm_size - sizeof(data_length)) {
    fprintf(stderr, "Invalid data length\n");
    goto cleanup;
}
```

```
input = malloc((size_t)data_length + 1);


if (input == NULL) {
    perror("malloc error");
    goto cleanup;
}


memcpy(input,
       shm_ptr + sizeof(data_length),
       (size_t)data_length);


input[data_length] = '\0';


int64_t sum = 0;
char *ptr = input;


while (*ptr != '\0') {


    while (isspace((unsigned char)*ptr)) {
        ptr++;
    }


    if (*ptr == '\0') {
        break;
    }
```

```
        errno = 0;
        char *end;


        long number = strtol(ptr, &end, 10);


        if (ptr == end || errno == ERANGE ||
            number < INT_MIN || number > INT_MAX) {
            fprintf(stderr, "Invalid integer: %s\n", ptr);
            goto cleanup;
        }


        if (*end != '\0' &&
            !isspace((unsigned char)*end)) {
            fprintf(stderr, "Invalid input near: %s\n", end);
            goto cleanup;
        }


        if ((number > 0 && sum > INT64_MAX - number) ||
            (number < 0 && sum < INT64_MIN - number)) {
            fprintf(stderr, "Sum overflow\n");
            goto cleanup;
        }


        sum += number;
        ptr = end;
    }


    FILE *file = fopen("result.txt", "w");
```

```
    if (file == NULL) {
        perror("fopen error");
        goto cleanup;
    }

    if (fprintf(file, "%" PRId64 "\n", sum) < 0) {
        perror("fprintf error");
        fclose(file);
        goto cleanup;
    }

    if (fclose(file) == EOF) {
        perror("fclose error");
        goto cleanup;
    }

    printf("Клиент: сумма = %" PRId64 "\n", sum);
    printf("Клиент: результат записан в result.txt\n");

    exit_status = EXIT_SUCCESS;

cleanup:
    free(input);

    if (sem != SEM_FAILED) {
        sem_close(sem);
    }
```

```
    if (shm_ptr != MAP_FAILED) {
        munmap(shm_ptr, shm_size);
    }


    if (shm_fd != -1) {
        close(shm_fd);
    }


    return exit_status;
}
```

Протокол работы программы

```
10 0 -9 7
Сумма = 8
```

Вывод

В ходе выполнения данной лабораторной работы я научилась работать с процессами и передавать данные между ними с помощью разделяемой памяти и технологии отображения файлов. В самом начале было немного сложно разобраться, как именно работает shm_open и как связать родительский и дочерний процессы, чтобы они видели одни и те же данные. Также пришлось повозиться с семафорами, чтобы клиент не начинал читать память раньше, чем сервер туда что-то запишет.

В результате программа успешно работает: родитель считывает строку с числами, кладет ее в общую память, а дочерний процесс через fork и execv подключается к этой памяти, считает сумму чисел и записывает ответ в файл result.txt. Я поняла, что использование shared memory и mmap это гораздо более быстрый способ обмена данными между процессами, чем обычные файлы или каналы, так как данные передаются напрямую через оперативную память. Лабораторная показалась интересной, хоть и потребовала внимательности при написании кода на Си.